What makes a difference when adding encapsulation to both the salary and contract?

Looking at the code you shared, you have actually already built the perfect foundation for **Encapsulation**. By marking your variables as `private` (like `private double baseSalary;`) and providing `public` methods (like `getStartDate()` and `setPerformanceBonus()`), you have successfully encapsulated both classes.

When you add encapsulation to `Salary` and `Contract`, it makes a massive difference in three practical ways:

1. It Acts as a Protective Shield (Data Guarding)

Without encapsulation, your variables would be `public`. That means any other file (like `ContractList`) could accidentally reach inside and change your data directly with a bad value, bypassing all rules.

- **Without Encapsulation (Dangerous):**

```java
// Any file can force a player's salary to be a impossible negative number
someSalary.baseSalary = -50000.0;
```

- **With Encapsulation (Safe):**
  Because `baseSalary` is `private`, outside code is forced to go through your rules. You control the gate. You can write a setter or use your validation methods to say: *"If the number is less than 0, reject it!"*

2. It Keeps Interdependent Data Synchronized

This is beautifully demonstrated in your `Salary.java` code. Notice what happens inside your `setPerformanceBonus` method:

Java

```java
public void setPerformanceBonus(double performanceBonus) {
    this.performanceBonus = performanceBonus;
    this.totalSalary = baseSalary + performanceBonus; // <-- The Magic Line
}
```

Because of encapsulation, the outside program doesn't need to remember to recalculate the total salary whenever a bonus changes. The `Salary` class handles its own internal math behind closed doors. The moment a new bonus is set, `totalSalary` updates automatically. Outside files just call `getTotalSalary()` and are guaranteed to get the correct, freshly calculated answer.

3. It Makes Your Code Modular ("The Black Box" Effect)

Encapsulation allows your management class (`ContractList`) to remain incredibly simple. `ContractList` does not need to understand how a contract determines its validity; it just asks the contract for the answer.

Look at your validation method inside `Contract.java` :

Java

```java
public boolean validateContract() {
    return baseSalary > 0 && status != null && !status.isEmpty();
}
```

When `ContractList` wants to check a contract, it simply runs:

Java

```java
if (newContract.validateContract()) { ... }
```

If your teacher changes the rules tomorrow and says, *"Contracts now also require the playerType to be 'Pro' to be valid,"* you **only** have to modify the code inside `Contract.java`. Your `ContractList.java` file doesn't have to change a single line of code because it never cared *how* the validation worked in the first place—it just trusted the `Contract` class to return `true` or `false`.

Summary for your Project

Encapsulation turns `Salary` and `Contract` from dumb, unprotected text containers into **smart, independent components** that guard their own data, calculate their own math, and handle their own rules safely away from outside interference.